Best Practices for Programming with AI:

A Practitioner's Guide

Jerod W. Wilkerson

February 2026

AI Attribution

Written with Claude Opus 4.6 by Anthropic and ChatGPT (GPT-5.2) by OpenAI

Who This Guide Is For

This guide is for computer science students and early-career developers who are starting to use AI coding tools—GitHub Copilot, Cursor, Claude Code, Codex, Google Antigravity, and others—and want to use them well. “Well” means more than just getting code out of a model. It means understanding what these tools cost, how to keep their output from breaking your codebase, how to run multiple AI agents without them stepping on each other’s work, and how to validate everything before it reaches production. This guide also addresses the academic integrity questions that come with using AI tools in coursework, grounded in BYU’s Academic Honesty Policy.

The core idea is simple: AI tools generate code probabilistically, but software engineering demands determinism. Every technique in this guide—diff-only workflows, Git worktree isolation, multi-agent orchestration, iteration caps, CI/CD gating—exists to bridge that gap. You will also find pricing comparisons, tool configuration examples, and ready-to-use prompts for Cursor and Claude Code.

You do not need deep AI experience to use this guide. You do need familiarity with Git, a terminal, and at least one programming language.

Who This Guide Is For 1
1. Introduction 1
2. Ethical and Academic Integrity Considerations 2
2.1 Academic Honesty and AI-Generated Code 2
2.2 Attribution and Disclosure 3
2.3 Learning Acceleration vs. Learning Bypass 5
2.4 Professional Considerations 5
3. Detailed Tool Landscape and Comparison 5
3.1 GitHub Copilot 6
3.2 Cursor 7
3.3 Claude Code 8
3.4 OpenAI Codex 9
3.5 Google Antigravity 10
3.6 Consolidated Comparison 12
4. Cost Awareness for Students 13
4.1 Pricing Paradigms 13
4.2 Worked Cost Example: How Iterative Loops Multiply Cost 13
4.3 Model Tiering Strategy 14
4.4 Cost Mitigation Techniques 14
5. Developer Productivity Architecture 15
5.1 Why Diff-Only Workflows Are Critical 15
Applying Diff-Only Workflows in Cursor 16
Applying Diff-Only Workflows in Claude Code 16
5.2 Structured Prompting Techniques 16
Example: Configuring Persistent Context in Cursor 17
Example: Configuring Persistent Context in Claude Code 17
Example: Configuring Persistent Context in Codex 18
Example: Configuring Persistent Context in Antigravity 18
5.3 Session Hygiene and Daily Workflow Habits 19
5.4 Debugging and Troubleshooting AI-Generated Code 20
Common Failure Patterns 20
Diagnostic Strategies 21
When to Abandon and Start Over 22
6. Git Worktrees for AI Concurrency 23
6.1 Why Worktrees Are Essential for AI Concurrency 23
6.2 Branch-Per-Agent Strategy 23
6.3 Parallel Agent Execution and Tool Setup 24
Setting Up Claude Code in a Worktree 24

Setting Up Cursor in a Worktree..... 25
Example Prompt for Worktree-Based Agent Work..... 25
6.4 Cleanup Strategy..... 25
7. Multi-Agent Orchestration Architecture..... 26
7.1 Agent Roles and Responsibilities..... 26
7.2 DAG-Based Execution Model..... 27
7.3 Iteration Caps and Escalation..... 28
8. Overwrite Prevention Techniques..... 29
8.1 Branch and Worktree Isolation..... 29
8.2 Read-Before-Write Prompt Enforcement..... 29
8.3 Diff-Only Enforcement and CI Rejection..... 29
8.4 Infinite Loop Prevention..... 30
9. CI/CD Integration..... 30
10. Conclusion..... 31
11. Resources..... 31
Appendix A: Tool and Model Version Reference..... 33
Appendix B: Multi-Agent Role Prompt Templates..... 34
Planner Prompt..... 34
Implementer Prompt..... 34
Tester Prompt..... 34
Verifier Prompt..... 35
Appendix C: Glossary of Key Terms..... 36

1. Introduction

Imagine you ask an AI coding tool to write a login function. The code it produces looks clean, passes the linter, and handles edge cases you hadn't thought of. But when you look more carefully, you notice it stores passwords in plain text. Nothing flagged the problem—the code is syntactically perfect. This is the core challenge of AI-assisted development: the output looks right, but "looks right" is not the same as "is right." This guide gives you the tools to tell the difference.

Software engineering has been built on the principle of determinism. A compiled binary, given the same inputs and the same execution environment, produces the same outputs. Version control systems record every change as an immutable, content-addressable object. Continuous integration pipelines enforce a fixed sequence of compilation, linting, testing, and deployment. Every layer of the modern software delivery lifecycle is designed to minimize entropy and maximize reproducibility. When a developer writes a function, the compiler either accepts it or rejects it; there is no probabilistic middle ground.

Large language models (LLMs) disrupt this deterministic foundation. An LLM is, at its core, a next-token prediction engine: given a prompt, it samples from a probability distribution over the vocabulary to produce each successive token. The same prompt submitted twice may yield different outputs, depending on temperature settings, sampling strategies, and model version. This introduces a category of risk that traditional software engineering tools were not designed to manage. A function generated by an LLM may be syntactically correct and pass type checks yet contain subtle logical errors, hallucinated API calls, or security vulnerabilities that no compiler will catch. The failure mode is not a crash—it is silent incorrectness.

The risk model change is substantial. In traditional development, risks cluster around human error during code authoring and are mitigated by peer review, static analysis, and automated testing. When an LLM generates code, the risk surface expands: in addition to the original human-error risks, developers must contend with hallucinated function signatures, fabricated library imports, non-existent API endpoints, subtly wrong algorithmic logic, and style drift that degrades codebase coherence over time. Furthermore, LLM outputs lack a human author whose mental model can be interrogated during code review.

These risks do not argue against using AI in software development; the productivity gains are too significant to ignore. Rather, they argue for architectural guardrails: deterministic structures that constrain the probabilistic output of LLMs before it enters the production codebase. This paper presents a comprehensive framework for such

guardrails, covering tool selection, cost awareness, diff-based workflows, Git worktree isolation, multi-agent orchestration, overwrite prevention, and CI/CD integration. The central thesis is straightforward: AI increases developer productivity only when embedded within deterministic guardrails that validate, isolate, and control every AI-generated artifact before it touches the main branch.

For students and junior developers, understanding these guardrails is arguably more important than understanding any individual AI tool. Tools will evolve rapidly—the specific features of Copilot, Cursor, Claude Code, Codex, and Antigravity today may look quite different a year from now. But the need for deterministic validation of probabilistic output is a permanent architectural concern. The patterns described in this paper apply regardless of which tool you use or which model powers it. Readers unfamiliar with terms such as LLM, context window, or hallucination will find definitions for these and other key terms in Appendix C.

2. Ethical and Academic Integrity Considerations

The tools described in this paper can generate, refactor, and debug code faster than any human. But the fact that you can use a tool does not mean you should—at least not in every context. Before integrating AI coding tools into your workflow, you need to understand the institutional and professional boundaries that govern their use.

2.1 Academic Honesty and AI-Generated Code

BYU's Academic Honesty Policy (https://policy.byu.edu/view/academic-honesty-policy) requires that students "complete their own work and be evaluated based upon that work." The policy categorizes violations as plagiarism, fabrication or falsification, cheating, and other academic misconduct—and AI-generated code can implicate all four. Submitting AI-generated code as your own without disclosure meets the policy's definition of plagiarism: "the deliberate act of representing the words, ideas, or data of another as one's own without providing proper attribution." Using AI to produce a solution you could not have written yourself creates "the appearance of a level of knowledge or skill that the student has not obtained"—the definition of cheating.

BYU does not maintain a campus-wide AI use policy. The university's position is that AI use should be "consistent with the principles of the Honor Code and the Academic Honesty Policy," with individual departments and instructors setting specific rules. The BYU Computer Science department does not publish a department-level AI policy; instead, policies are set at the course level and vary significantly. CS 111, for example, explicitly prohibits AI-assisted autocompletion tools—including GitHub Copilot and ChatGPT—on all assignments and progress checks. CS 340 allows AI to be used as a tutor (but not a coding assistant) on all assignments, and encourages unrestricted AI

use on some assignments during the second half of the semester (see syllabus for details). The BYU GenAI guidance to students is clear: “Before using GenAI on an assignment, you must ask your professor for permission if they have not already explicitly stated that it is permitted. If the professor has not said anything about GenAI use for either the assignment or in the syllabus, assume that using GenAI is not permitted.”

The practical rule is simple: check the syllabus for every course, and when in doubt, ask your instructor before using any AI tool on any assignment.

2.2 Attribution and Disclosure

When AI use is permitted, proper attribution is required. When you use an AI code-generation tool, add a comment identifying which portions were AI-generated, which tool and model produced them, and what prompt you used:

```
// Generated by Claude Code (Claude Sonnet 4.5) using prompt:
// "Write a middleware function that rate-limits requests
//  to 100 per minute per IP using a sliding window."
// Reviewed and modified by [Your Name] on [Date].
```

Attribution is not merely a compliance exercise. It allows your instructor to evaluate which portions of the work represent your understanding, and it creates a record that protects you if questions arise later.

For large, multi-file projects where AI assistance is used extensively, per-line comments become impractical. In these cases, maintain a project-level attribution file—an AI-ATTRIBUTION.md in the repository root—that serves as a centralized disclosure log. Each entry should record the date, the file(s) affected, the tool and model used, the prompt or task description, and a brief note on what you reviewed or modified afterward. For example:

```
# AI Attribution Log

## 2026-02-20 — src/auth/rateLimit.ts (new file)
- Tool: Claude Code (Claude Sonnet 4.5)
- Prompt: "Implement sliding-window rate limiting middleware
   for Express, 100 requests/min per IP, with Redis backing."
- Review: Rewrote Redis connection handling; added error
   fallback for Redis downtime. All tests written manually.

## 2026-02-21 — src/auth/rateLimit.test.ts (new file)
- Tool: GitHub Copilot (inline completions)
- Prompt: Autocomplete suggestions accepted for test scaffolding.
- Review: Verified assertions and added edge-case tests for
   concurrent requests. Approximately 40% Copilot-suggested,
   60% manually written.

## 2026-02-22 — src/db/schema.ts (modified, lines 45-72)
```

```
- Tool: Cursor Agent (Claude Sonnet 4.5)
- Prompt: "Add a rateLimitEntry table with columns for IP,
   window start timestamp, and request count."
- Review: Accepted as generated; verified migration runs cleanly.
```

Individual files should still include a brief header comment noting that AI was used—something as simple as // AI-assisted — see AI-ATTRIBUTION.md for details—so that reviewers encountering a single file know to check the log. This two-tier approach (project-level log plus lightweight file-level markers) scales to projects of any size while keeping the attribution thorough and auditable.

You can configure your AI tools to maintain this log automatically. In Claude Code, add the following to your project's CLAUDE.md file:

```
# CLAUDE.md (add to existing file)
## Attribution Logging
- After completing any task that creates or modifies code files,
  append an entry to AI-ATTRIBUTION.md in the project root.
- Each entry must include:
  - Date and files affected (with "new file" or line ranges)
  - Tool and model (e.g., "Claude Code (Claude Sonnet 4.5)")
  - The prompt or task description that initiated the work
  - A brief summary of what was generated vs. what the developer
    reviewed or modified
- Add a header comment to each created or modified file:
  // AI-assisted — see AI-ATTRIBUTION.md for details
- If AI-ATTRIBUTION.md does not exist, create it with a
  top-level heading: # AI Attribution Log
```

In Cursor, add the equivalent to your .cursorrules file (or a file in .cursor/rules/):

```
# .cursorrules (add to existing file)
## Attribution Logging
- After completing any task that creates or modifies code files,
  append an entry to AI-ATTRIBUTION.md in the project root.
- Each entry must include:
  - Date and files affected (with "new file" or line ranges)
  - Tool and model (e.g., "Cursor Agent (Claude Sonnet 4.5)")
  - The prompt or task description that initiated the work
  - A brief summary of what was generated vs. what the developer
    reviewed or modified
- Add a header comment to each created or modified file:
  // AI-assisted — see AI-ATTRIBUTION.md for details
- If AI-ATTRIBUTION.md does not exist, create it with a
  top-level heading: # AI Attribution Log
```

These instructions cause the tool to update the log as part of its normal workflow, so attribution happens automatically rather than relying on the developer to remember after the fact. You should still review each log entry for accuracy—the tool may summarize its own changes imprecisely—but the overhead drops from writing entries manually to spot-checking entries the tool has drafted for you.

2.3 Learning Acceleration vs. Learning Bypass

Attribution addresses transparency—ensuring that others know how your code was produced. But transparency alone is not sufficient. A student can attribute every line of AI-generated code perfectly and still undermine their own education if they never engaged with the material. The deeper question is whether you are using AI to *accelerate* your learning or to *bypass* it.

Using AI as a learning accelerator means attempting a problem yourself, getting stuck, asking the AI to explain a concept, then applying that understanding to write your own solution. It means reading every line of AI-generated code before incorporating it and asking the model why it chose a particular approach. Using AI as a learning bypass means pasting an assignment prompt into a tool, accepting the output without understanding it, and submitting it. No learning occurs, and the grade ceases to reflect your competence.

A useful self-test: *Could I explain this code line by line to a classmate? Could I write something equivalent without the tool? Could I debug it if it broke?* If the answer to any of these is no, you have bypassed learning and should go back and fill the gap.

2.4 Professional Considerations

Three professional concerns extend beyond the classroom. First, **intellectual property**: AI models are trained on code published under various open-source licenses, and whether AI-generated output constitutes a derivative work is legally unsettled. Be aware of this uncertainty, especially in open-source or commercial work. Second, **confidentiality**: submitting code to an AI service sends it to a third-party server. BYU's GenAI guidance warns that "nonpublic institutional data should never be entered into a GenAI service." Build the habit now of being deliberate about what you send to AI tools. Third, **accountability**: when AI-generated code causes a bug or security vulnerability in production, the developer who committed it is responsible—not the AI. This principle reinforces the paper's central thesis: every line of AI-generated code must be reviewed, tested, and validated by a human who takes ownership of it.

3. Detailed Tool Landscape and Comparison

The AI-assisted development tool market has rapidly diversified. Rather than a single dominant tool, practitioners now face a spectrum of offerings that differ along axes of context depth, agent autonomy, pricing model, and integration philosophy. This section surveys five major tools: GitHub Copilot, Cursor, Claude Code, Codex, and Antigravity. Understanding their differences is essential for selecting the right tool for a given workflow and for designing guardrails that account for each tool's specific risk profile.

3.1 GitHub Copilot

GitHub Copilot, developed by GitHub in partnership with OpenAI, is an inline code completion engine available as an extension for Visual Studio Code, JetBrains IDEs, and other editors. As you type, Copilot predicts the next lines of code and offers them as ghost text that can be accepted with a single keystroke. Beyond autocomplete, Copilot provides a chat interface (Copilot Chat) for more complex prompts, an agent mode for multi-step tasks, and a coding agent that can work asynchronously on GitHub issues.

Copilot's strengths lie in its low friction and broad language support. It handles boilerplate generation, test scaffolding, and documentation comments exceptionally well. Because it is backed by OpenAI's GPT-5 family models (with access to additional models like Claude on higher tiers), its code quality is generally strong for single-function completions. However, Copilot's context window is limited by what is currently open in the editor; it does not deeply index the full repository. This means that cross-file reasoning—understanding how a utility function in one module is consumed by another—can be shallow compared to tools that index the entire codebase. Additionally, Copilot's multi-agent capabilities are limited; it operates primarily as a single-agent system with some agentic mode features for issue resolution.

Copilot uses a seat-based subscription model with premium request metering for advanced features. Students, educators, and verified open-source maintainers receive Copilot Pro access at no cost through the GitHub Student Developer Pack, making it the most accessible tool for university students. The risk level is generally low because Copilot's suggestions are typically small in scope—inline completions and single-function generation—limiting the blast radius of any hallucinated output.

Table 1: GitHub Copilot Pricing Tiers (as of February 2026)

Tier	Price	Billing	Key Features
Free	$0/month	N/A	2,000 completions/mo, 50 premium requests/mo
Pro	$10/month	Monthly	Unlimited completions, 300 premium requests/mo, Copilot Chat
Pro+	$39/month	Monthly	1,500 premium requests/mo, all models incl. Claude Opus & o3
Business	$19/seat/mo	Monthly	Org management, IP indemnity, policy controls
Enterprise	$39/seat/mo	Monthly	1,000 premium req/seat, knowledge bases, custom models

Note: Premium requests are consumed by Copilot Chat, agent mode, coding agent, and code review features. Different models consume premium requests at different rates; a single query to a top-tier model may cost 50 premium requests. Overage pricing is $0.04 per additional premium request.

3.2 Cursor

Cursor is an AI-native code editor forked from Visual Studio Code that treats AI as a first-class citizen of the editing experience rather than an add-on extension. Cursor maintains a codebase index that provides deeper repository-level context than extension-based tools. Its key differentiators include an Agent mode for multi-file edits, codebase-aware chat, integrated terminal AI, and background agents that can run in isolated cloud environments. Cursor supports multiple backend models, including Claude Sonnet 4.5, GPT-5.3-Codex, Gemini 3 Pro, and Cursor's own Composer-1 model, giving users the ability to switch between model providers based on task requirements.

Cursor's pricing model underwent significant changes in mid-2025, shifting from a request-based system to a credit-based model. Each paid plan includes a monthly credit pool (denominated in dollars) that depletes at different rates depending on which AI model you select and how much context is included. Using the "Auto" model—which Cursor selects—is unlimited on paid plans, while manually selecting premium models like Claude Opus consumes credits at their respective API rates. This hybrid model provides flexibility but has generated confusion; users have reported unexpected bills when credit pools were depleted faster than anticipated. For teams that rely heavily on premium models, this can become a significant cost factor.

Cursor's risk profile is moderate. Its Agent mode helps contain changes by showing diffs inline before they are applied, and you can accept or reject changes at the file level. However, users must still review multi-file edits carefully to catch hallucinated imports or unintended side effects. Cursor's strongest advantage for students is its familiar VS Code interface and its ability to work with the full repository context, making it effective for feature-level work that spans multiple files.

A key configuration mechanism in Cursor is the .cursorrules file (or the newer .cursor/rules/ directory), which contains persistent instructions that are injected into every AI interaction within the project. This allows teams to codify project conventions—such as preferred frameworks, naming conventions, testing patterns, and forbidden patterns—so that the AI respects them automatically. We will discuss this configuration in detail in Section 5.

Table 2: Cursor Pricing Tiers (as of February 2026)

Tier	Price	Billing	Key Features
Hobby (Free)	$0/month	N/A	Limited Agent requests and Tab completions
Pro	$20/month	Monthly/Annual	Unlimited Tab & Auto, $20/mo credit pool for premium models
Pro+	$60/month	Monthly/Annual	$60/mo credit pool (3x Pro), same unlimited Auto
Ultra	$200/month	Monthly/Annual	~20x Pro usage pool, priority feature access
Teams	$40/seat/mo	Monthly	SSO, admin controls, per-user usage accounting
Enterprise	Custom	Annual	SCIM, audit logs, pooled credits, custom controls

Note: Credits deplete at different rates by model. Claude Opus consumes credits approximately 2.4x faster than Gemini models from the same dollar pool. Annual billing saves approximately 20%.

3.3 Claude Code

Claude Code, developed by Anthropic, is a command-line interface (CLI) tool that provides agentic coding capabilities directly in the terminal. Unlike editor-integrated tools, Claude Code operates at the project level: it can read, modify, and create files across an entire repository, run shell commands, execute tests, and iterate on failures—all within a single conversation session. Claude Code uses Anthropic's Claude models (Sonnet and Opus tiers) and is available through multiple pricing paths: the Anthropic API with token-based billing, the Claude Pro subscription ($20/month) with usage limits, or the Claude Max subscription ($100–$200/month) with substantially higher limits.

Claude Code's strength is its deep agentic capability. It can plan a multi-step implementation, execute it, run verification commands, and iterate on errors without human intervention between steps. Recent additions include a VS Code extension, a desktop application, and Agent Teams (on Opus 4.6) that allow multiple sub-agents to work in parallel on different aspects of a task. The 200K token context window (1M in beta on Opus 4.6) means Claude Code can reason about large codebases without the context truncation issues that affect some editor-based tools. Independent testing has shown Claude Code to be notably token-efficient, using significantly fewer tokens than competing tools for equivalent tasks.

However, this autonomy is also its primary risk factor. An unconstrained Claude Code session can modify many files in rapid succession, potentially introducing cascading errors or overwriting concurrent work. The token-based cost model also means that

long, iterative sessions with large codebases can become expensive; a single complex task involving Opus-tier models can consume substantial token budgets. For teams, Claude Code is best deployed within the worktree isolation and diff-only workflows described later in this paper.

Claude Code uses a CLAUDE.md file as its persistent context configuration. Placing a CLAUDE.md file in the project root allows developers to specify project conventions, build commands, test procedures, and coding standards that persist across all Claude Code sessions. This is analogous to Cursor's .cursorrules file and plays a critical role in the structured prompting techniques discussed in Section 5.

Table 3: Claude / Claude Code Pricing Tiers (as of February 2026)

Tier	Price	Billing	Key Features
Free	$0/month	N/A	Limited Sonnet access, no Claude Code
Pro	$20/month	Monthly/Annual	5x Free usage, Claude Code access, Sonnet 4.6 default
Max 5x	$100/month	Monthly	5x Pro usage, Opus 4.6 access, 1M context beta
Max 20x	$200/month	Monthly	20x Pro usage, zero-latency priority, full Opus access
Team (Std)	$25/seat/mo	Annual	SSO, admin, Slack/M365 integrations, no Claude Code
Team (Prem)	$150/seat/mo	Annual	All Team features plus Claude Code and early features
Enterprise	Custom	Annual	Full governance, compliance, dedicated support

Note: Claude Code on Pro may hit usage limits during sustained coding sessions. The Max plans are recommended for developers using Claude Code as a primary daily tool. API pricing (pay-per-token) is also available: Haiku 4.5 at $1/$5, Sonnet 4.5 at $3/$15, and Opus 4.5/4.6 at $5/$25 per million input/output tokens.

3.4 OpenAI Codex

OpenAI Codex is an agentic coding tool available in two distinct modes. The Codex CLI is an open-source, terminal-based agent written in Rust that runs locally on your machine, reading files, proposing edits, running commands, and iterating on results at the project level. Codex Web is a cloud-based agent accessed through ChatGPT that executes tasks asynchronously in isolated sandbox environments preloaded with your repository. Task completion typically takes between 1 and 30 minutes, after which you can review results, request revisions, or open a GitHub pull request. Both modes are powered by GPT-5.3-Codex, OpenAI's latest model optimized for software engineering tasks, and are connected by your ChatGPT account.

In its CLI form, Codex is closely comparable to Claude Code: both are terminal-first agentic tools that operate at the repository level with a persistent context file — AGENTS.md in Codex, CLAUDE.md in Claude Code. The meaningful differentiator is Codex's cloud execution mode, which allows longer-running tasks to be delegated asynchronously without blocking local development. The CLI offers three approval modes: Auto (read, edit, and run within the working directory), Read-only (consultative browsing only), and Full Access (unrestricted execution including network calls). This tiered approval system gives developers more explicit control over agent autonomy than most competing tools.

Codex is not sold as a standalone product; access is included with ChatGPT subscriptions. Students already paying for ChatGPT Plus ($20/month) gain Codex access at no additional cost — a meaningful advantage for those already in the OpenAI ecosystem. Unlike Copilot, however, there is no dedicated student discount or free tier for Codex in 2026, making it less accessible for students who do not already subscribe to ChatGPT. Token-based API access is available via the codex-mini-latest model for developers who prefer pay-per-token billing.

Codex's risk profile is high, comparable to Claude Code, due to its deep agentic autonomy. The cloud execution mode introduces an additional confidentiality consideration — repository code is transmitted to and executed on OpenAI's infrastructure — which warrants the same caution noted in Section 2.4 for any tool that sends code to a third-party server.

Table 4: OpenAI Codex Pricing Tiers (as of February 2026)

Tier	Price	Billing	Key Features
Plus	$20/month	Monthly	Codex CLI + Web, usage-limited (30-150 msgs/5 hrs)
Pro	$200/month	Monthly	Higher limits (300-1,500 msgs/5 hrs), GPT-5.3-Codex-Spark access
Business	$30/seat/month	Monthly	Team controls, usage accounting, admin dashboard
Enterprise	Custom	Annual	Audit logs, compliance, custom controls
API (codex-mini-latest)	Pay-per-token	Usage	$1.50/$6.00 per million input/output tokens

Note: Codex is not a standalone product; access requires an existing ChatGPT subscription or OpenAI API key. The CLI is open source and free to install. No student discount is available in 2026; students without an existing ChatGPT subscription must pay the standard Plus rate. Usage limits on Plus and Pro plans vary based on message complexity and token consumption.

3.5 Google Antigravity

Google Antigravity, announced in November 2025 alongside the release of Gemini 3, represents the agent-first IDE category. Antigravity is a VS Code–based editor

redesigned around autonomous agent workflows. Its distinguishing feature is the Agent Manager view, a mission-control interface where you can dispatch multiple agents to work on different tasks simultaneously, each operating across the editor, terminal, and a built-in browser. Agents produce artifacts—task plans, implementation plans, diffs, screenshots, and browser recordings—that serve as verifiable deliverables for human review.

Antigravity provides the deepest integration between agent and development environment of any tool surveyed here. Agents can write code in the editor, execute shell commands in the terminal, launch applications, and drive a browser to verify UI changes—all autonomously. It supports multiple AI models, including Gemini 3 Pro (default), Gemini 3 Deep Think, Gemini 3 Flash, Claude Sonnet 4.5, and GPT-OSS. The 1M-token context window of Gemini 3 Pro allows Antigravity to ingest entire large codebases without truncation.

Antigravity also features a knowledge base that learns from conversations, effectively building project-specific context that improves over time. The permission system provides granular control over agent autonomy, from “Always proceed” (full autonomy) to “Request review” (approval required for every action). This permission system is essential because Antigravity’s agents have the broadest scope of action of any tool surveyed: they can modify files, run arbitrary shell commands, and interact with web browsers.

Antigravity supports persistent context through a Rules system analogous to Cursor's .cursorrules file. Workspace rules are Markdown files stored in the .agent/rules/ directory at your project root; Antigravity loads them automatically into every agent session, injecting your coding standards and project conventions without requiring you to repeat them in each prompt. Global rules, stored in ~/.gemini/GEMINI.md, apply across all projects. We will discuss this configuration in detail in Section 5.

At the time of writing, Antigravity is available at no cost during its public preview, with generous rate limits on Gemini 3 Pro usage. Post-preview pricing has not been announced. This makes Antigravity attractive for students who want to experiment with agent-first workflows, but the lack of established pricing creates uncertainty for long-term planning. The risk profile is the highest of any tool surveyed because of the broad scope of agent autonomy and the relative immaturity of the platform’s documentation and community best practices.

Table 5: Google Antigravity Pricing (as of February 2026)

Tier	Price	Billing	Key Features
Public Preview	$0	N/A	All features, generous Gemini 3 Pro rate limits
Post-Preview	TBD	TBD	Pricing not yet announced

Note: Antigravity is currently free during public preview. Students should take advantage of this window to experiment, but should not build critical workflows around a tool whose long-term pricing is unknown.

3.6 Consolidated Comparison

Table 6 consolidates the key characteristics of each tool across the dimensions most relevant to students and engineering teams making adoption decisions. No single tool is universally superior; the appropriate choice depends on the task's scope, the developer's risk tolerance, and the guardrail infrastructure already in place.

Table 6: Tool Comparison Summary

Tool	Strength	Multi-Agent	Pricing Model	Best Use Case	Risk Level
Copilot	Low friction, broad language support	Limited (agent mode)	Seat + premium requests ($0–39/mo)	Boilerplate, single-function tasks	Low
Cursor	Deep repo context, multi-model	Agent mode, background agents	Credit pool + overages ($0–200/mo)	Multi-file feature work	Moderate
Claude Code	Deep agentic capability, token-efficient	Agent Teams (Opus 4.6)	Subscription or API tokens ($0–200/mo)	Complex refactoring, agentic tasks	High
Codex	Dual local/cloud modes, open-source CLI	Experimental multi-agent (CLI) + parallel cloud tasks	Subscription or API tokens ($20–200/mo)	Async cloud delegation, local agentic tasks	High
Antigravity	Editor + terminal + browser agents	Multi-agent orchestration	Free (preview); TBD	Greenfield, full-stack agent work	Highest

As Table 6 illustrates, a team with robust worktree isolation and CI gates can safely leverage higher-autonomy tools like Claude Code or Antigravity; a team without such infrastructure should start with lower-risk tools like Copilot and build guardrails incrementally. For students, the recommendation is to begin with Copilot (free via the Student Developer Pack) or the free tiers of Cursor and Claude, then graduate to higher-autonomy tools as you develop comfort with the validation workflows described

in the remaining sections. For a snapshot of specific tool and model versions current at the time of writing, see Appendix A.

4. Cost Awareness for Students

AI coding tools introduce variable costs that scale with usage intensity, model selection, and workflow design. Unlike a traditional IDE license with a fixed annual cost, AI tools can generate costs that fluctuate significantly from month to month. You need to understand three pricing paradigms to make informed decisions.

4.1 Pricing Paradigms

Seat-based pricing charges a flat monthly fee per seat regardless of usage. GitHub Copilot's Pro plan exemplifies this model: you pay $10/month whether you use Copilot once or a thousand times. The advantage is budget predictability; the disadvantage is that you pay the same amount even in months of light usage. Credit-based pricing allocates a fixed pool of credits per billing cycle. Cursor's Pro plan includes $20 worth of premium model usage; once exhausted, you switch to the unlimited "Auto" model or enables pay-as-you-go overages. Token-based pricing charges per input and output token consumed. Anthropic's API pricing is the canonical example: each API call costs a calculable amount based on the number of tokens sent and received, multiplied by the model's per-token rate.

4.2 Worked Cost Example: How Iterative Loops Multiply Cost

The following example applies to any situation where you are being billed per token — most commonly when using Claude Code via the Anthropic API directly, or when using a tool like Cursor or GitHub Copilot and your monthly credit or premium request allotment has been exhausted and usage rolls over to pay-as-you-go overages. Developers on flat subscription plans who stay within their allotment can skip this example, though understanding the underlying math is still useful for appreciating why iteration caps and context pruning matter.

For API users and anyone in overage territory, costs can grow quickly and in ways that are easy to underestimate. Consider a student using Claude Code via the Anthropic API with Claude Sonnet 4.5 ($3 per million input tokens, $15 per million output tokens). A typical coding task involves a prompt of approximately 4,000 input tokens and generates approximately 2,000 output tokens:

```
Cost_per_task = (4,000 / 1,000,000) * $3.00
              + (2,000 / 1,000,000) * $15.00
              = $0.012 + $0.030
              = $0.042 per task
```

At $0.042 per task and 20 tasks per day, the daily cost is $0.84—roughly $18.50 for a 22-day working month. This is manageable. However, agentic tools frequently iterate: the model generates code, runs tests, encounters a failure, reads the error, and retries. If each task averages three iterations instead of one, costs triple to $55.44 per month. If some tasks spiral into extended debugging loops of 8–10 iterations with larger context windows (20,000 input tokens), a single runaway task can cost $0.90 or more. Across a month of heavy development, uncontrolled iteration can push API costs into hundreds of dollars.

The lesson is clear: iteration is the primary cost multiplier. Every retry sends the full context back through the model, and costs compound linearly with iteration count.

4.3 Model Tiering Strategy

The most effective cost mitigation technique is model tiering: using cheaper models for simpler tasks and reserving expensive models for critical verification. For planning and task decomposition, use a fast, cheap model like Claude Haiku ($1/$5 per million tokens). For implementation, use the balanced mid-tier model like Claude Sonnet ($3/$15). For final verification and code review, use the most capable model like Claude Opus ($5/$25) but only on the final diff, keeping context small. By routing the majority of tokens through cheaper models, you can reduce your effective blended cost by 40–60%.

4.4 Cost Mitigation Techniques

Beyond model tiering, you should implement several cost controls. Set iteration caps: define a maximum number of retries per task (e.g., five) so that a failing task is escalated to manual debugging rather than burning tokens indefinitely. Use prompt caching: Anthropic's prompt caching feature can reduce repeated-prefix input costs by up to 90%, which is valuable when the system prompt and codebase context are re-sent with each iteration. Prompt caching is only available when making Claude API calls, not when typing prompts into a terminal. See (https://platform.claude.com/docs/en/build-with-claude/prompt-caching) for details. Prune context: strip irrelevant files from the prompt before submission to reduce input token counts. Set spending alerts: if using API-based pricing, configure alerts at thresholds (e.g., $10/day) so you are notified before costs escalate. Finally, take advantage of free tiers: Copilot is free for students, Cursor has a free Hobby tier, Claude has a free tier, Antigravity is currently free in preview, and Codex is included with existing ChatGPT Plus subscriptions. Start with these before committing to paid plans.

5. Developer Productivity Architecture

The core productivity architecture for AI-augmented development follows a six-stage pipeline, illustrated in Figure 1: Specification, Diff Generation, Worktree Isolation, Automated Validation, Iteration, and Controlled Merge. Each stage serves as a deterministic checkpoint that constrains the probabilistic output of the preceding stage, ensuring that AI-generated code passes through multiple validation layers before it reaches the main branch.

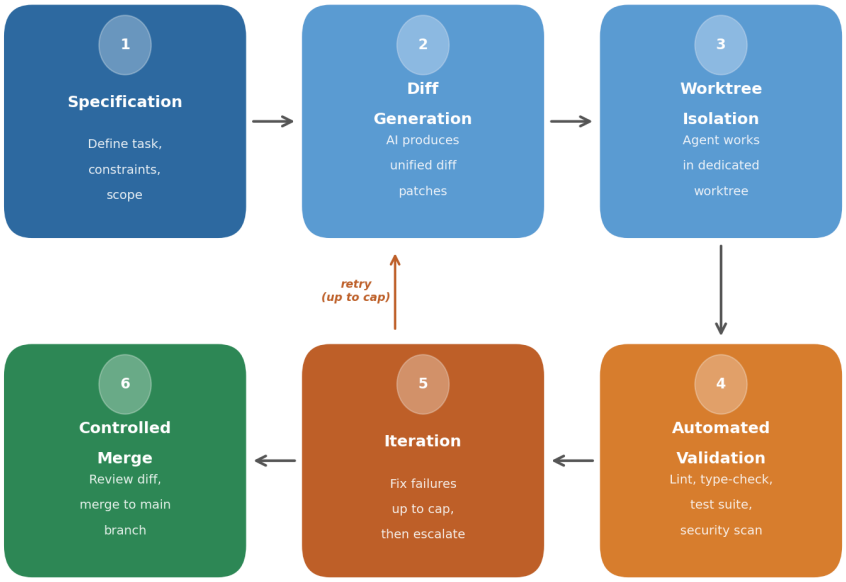

Figure 1: Six-Stage Developer Productivity Pipeline.

5.1 Why Diff-Only Workflows Are Critical

A diff-only workflow means that the AI agent is instructed to produce changes as unified diff patches rather than regenerating entire files. This is critical for several reasons. First, full-file regeneration destroys information: formatting conventions, manual comments, carefully tuned logic, and code that was not part of the requested change can all be silently overwritten. Second, diffs are reviewable; a reviewer can see exactly which lines were added, modified, or removed, which drastically reduces review time. Third, diffs are composable: multiple agents working on different parts of the codebase can each produce diffs that are merged independently, whereas full-file regenerations from concurrent agents will conflict destructively.

In practice, most modern AI coding tools default to diff-based workflows. Cursor's Agent mode shows proposed changes as inline diffs that you can accept or reject per file. Claude Code applies changes to files directly but can be configured (via CLAUDE.md instructions) to present diffs for approval before applying them. The key is to ensure your tool is configured to show you exactly what changed before those changes are committed.

Applying Diff-Only Workflows in Cursor

Cursor's Agent mode naturally produces diffs. When you issue a prompt in Agent mode, Cursor proposes changes across one or more files, displayed as inline additions and deletions. You can review each file's changes and accept or reject them individually. To enforce stricter diff discipline, add the following to your .cursorrules file:

```
# .cursorrules
Always show proposed changes as diffs before applying.
Never regenerate an entire file; only modify the specific
lines relevant to the task.
Preserve all existing comments, formatting, and imports
that are not directly related to the requested change.
```

Applying Diff-Only Workflows in Claude Code

Claude Code can be configured to operate in a diff-aware manner by adding instructions to the project's CLAUDE.md file. While Claude Code applies changes directly to files by default, you can instruct it to show diffs before committing:

```
# CLAUDE.md
## Code Modification Rules
- Before modifying any file, show the proposed diff and wait
  for confirmation.
- Never regenerate an entire file. Only modify lines directly
  relevant to the current task.
- Preserve all existing comments, formatting, and import
  ordering.
- After applying changes, run `git diff` to display the
  actual changes made.
```

5.2 Structured Prompting Techniques

Effective prompting for AI-assisted development requires explicit instruction about the format, scope, and constraints of the desired output. The agent must be told to produce output in a structured format, to include file paths, to stay within the scope of the task, and to preserve existing conventions. These instructions function as a soft contract between you and the agent. Without them, the agent may make well-intentioned but

undesirable changes—reformatting code, reordering imports, adding unsolicited improvements, or modifying files outside the scope of the request.

The most effective approach is to encode these instructions in a persistent configuration file that is automatically injected into every AI interaction. In Cursor, this is the .cursorrules file (or files in .cursor/rules/). In Claude Code, this is the CLAUDE.md file placed in the project root. Both serve the same purpose: they provide the AI with project-specific context that persists across prompts and sessions.

Example: Configuring Persistent Context in Cursor

Create a .cursorrules file in your project root with content like:

```
# .cursorrules
## Project Context
This is a TypeScript monorepo using Next.js 15, Prisma ORM,
and PostgreSQL. The test framework is Vitest.

## Coding Standards
- Use functional components with hooks (no class components).
- All API routes must include input validation with Zod.
- Use named exports, not default exports.
- Error handling: always use Result<T, E> pattern.

## Forbidden Patterns
- Do NOT use `any` type.
- Do NOT use console.log in production code (use the logger).
- Do NOT modify files in src/generated/ (these are auto-generated).
```

Example: Configuring Persistent Context in Claude Code

Create a CLAUDE.md file in your project root:

```
# CLAUDE.md
## Project Overview
TypeScript monorepo: Next.js 15 frontend, Express backend,
Prisma ORM, PostgreSQL. Tests use Vitest.

## Build & Test Commands
- Build: `npm run build`
- Test: `npm run test`
- Lint: `npm run lint`
- Type check: `npx tsc --noEmit`

## Conventions
- Functional components with hooks only.
- Zod validation on all API inputs.
- Named exports only. No default exports.
- Result<T, E> pattern for error handling.
- Never modify src/generated/ files.
```

When Claude Code starts a session in a directory containing CLAUDE.md, it reads and incorporates these instructions automatically. This means every prompt you issue within that project is augmented with your project conventions, build commands, and constraints—without you having to repeat them.

Example: Configuring Persistent Context in Codex

Create an AGENTS.md file in your project root:

```
# AGENTS.md
## Project Context

TypeScript monorepo: Next.js 15 frontend, Express backend,
Prisma ORM, PostgreSQL. Tests use Vitest.

## Build & Test Commands
- Build: `npm run build`
- Test: `npm run test`
- Lint: `npm run lint`
- Type check: `npx tsc --noEmit`

## Conventions
- Functional components with hooks only.
- Zod validation on all API inputs.
- Named exports only. No default exports.
- Result<T, E> pattern for error handling.
- Never modify src/generated/ files.
```

When Codex starts a session in a directory containing AGENTS.md, it reads and incorporates these instructions automatically, applying your project conventions to every agent interaction without requiring you to repeat them.

Example: Configuring Persistent Context in Antigravity

Create a GEMINI.md file in your project's .gemini/ directory, or add rules as Markdown files in .agent/rules/:

```
# .gemini/GEMINI.md
## Project Context
This is a TypeScript monorepo using Next.js 15, Prisma ORM,
and PostgreSQL. The test framework is Vitest.

## Coding Standards\

Use functional components with hooks (no class components).\
All API routes must include input validation with Zod.\
Use named exports, not default exports.\
Error handling: always use Result<T, E> pattern.

## Forbidden Patterns\
Do NOT use `any` type.\
Do NOT use console.log in production code (use the logger).\
Do NOT modify files in src/generated/ (these are auto-generated).
```

When Antigravity starts a session in a project containing these files, it reads and incorporates the rules automatically, applying your project conventions to every agent interaction without requiring you to repeat them.

5.3 Session Hygiene and Daily Workflow Habits

Sections 5.1 and 5.2 describe how to configure tools and structure prompts, but developer habits during daily use are equally important for maintaining control over AI-generated output. The following practices are simple but frequently overlooked, especially by developers new to AI-augmented workflows. Adopting them consistently prevents the class of problems that no tool configuration can solve on its own.

Start with a clean Git state. Before starting an AI agent session, commit or stash all uncommitted changes. This ensures that every modification in the working directory was made by the agent, making it straightforward to review changes with git diff and roll back entirely with git checkout . if needed. Without a clean starting state, agent-generated changes become tangled with in-progress human work, and selective rollback becomes difficult.

Synchronize context files across tools. If you use multiple AI tools on the same project—for example, Cursor for interactive editing and Claude Code or Codex for agentic tasks—maintain .cursorrules, CLAUDE.md, and AGENTS.md with shared project conventions. Put general context (architecture, coding standards, forbidden patterns) in all three files. Put tool-specific instructions—such as build and test commands for Claude Code, or UI behavior preferences for Cursor—only in the relevant file. Commit all of these files to the repository so they propagate to worktrees automatically (see Section 6). For larger teams, consider a script that syncs shared sections from a single source-of-truth file into .cursorrules, CLAUDE.md and AGENTS.md to prevent drift between them.

Scope tasks small and focused. Break work into single-purpose tasks: one function, one bug fix, one feature slice. Small tasks keep agent output reviewable, limit the blast radius if the agent goes off-track, and avoid context window degradation that occurs in long, multi-topic sessions. When you finish a task, commit the result and start a fresh session for the next one.

Checkpoint frequently with commits. Do not wait until a task is complete to commit. Commit after each meaningful increment—a passing test, a completed function, a successful refactor. If the agent produces a bad change in a later step, you can reset to the last good checkpoint rather than losing all progress. Use descriptive commit messages that note agent involvement (e.g., *Added rate limit middleware [claude-code]*), which helps reviewers identify AI-generated code in the Git history.

Start fresh sessions when context degrades. AI agents lose coherence as conversations grow long. If the agent begins repeating itself, forgetting earlier decisions, or producing increasingly generic output, end the session. Before starting a new one, write a brief handoff summary—what has been completed, what decisions were made, what remains—and provide it as the opening context for the new session. In Claude Code, the `/compact` command can compress context mid-session, but starting fresh is more reliable for extended tasks.

Review every diff before accepting. The temptation to accept AI-generated changes without careful reading grows as the agent produces consistently good output. Resist this. Treat every AI-generated diff with the same scrutiny you would apply to a pull request from a new team member: read every line, check for hallucinated imports or API calls, verify that unchanged code was not silently modified, and confirm that the change addresses the task you specified. Section 9 describes CI-level validation gates, but those gates are a backstop—they do not substitute for your own review at the point of accepting changes.

5.4 Debugging and Troubleshooting AI-Generated Code

Sections 5.1–5.3 describe how to structure, scope, and review AI-generated output. But even with well-configured guardrails, AI-generated code will sometimes fail—and debugging it is a distinct skill from debugging code you wrote yourself. When you write code, you carry a mental model of what each line is supposed to do; when an AI writes code, that mental model does not exist unless you actively build it. This section covers the common failure patterns in AI-generated code, practical strategies for diagnosing them, and guidance on when to abandon an AI-generated approach rather than iterating further.

Common Failure Patterns

AI-generated code fails differently from human-written code. Recognizing these patterns helps you diagnose problems faster instead of treating every bug as a generic logic error.

Hallucinated APIs and imports are the most distinctive AI failure mode. The model may call a function that does not exist in the library it imported, use a method signature that belongs to a different version of the API, or import a package that was never published. These errors often survive linting and type checking if the hallucinated name happens to match a valid type. When you see an ImportError, ModuleNotFoundError, or “method not found” at runtime, verify the import against the library’s official documentation before assuming your environment is misconfigured—the function itself may not exist.

Plausible-but-wrong logic is harder to catch because the code runs without errors and produces output that looks reasonable. Examples include off-by-one errors in loop bounds, incorrect operator precedence, reversed comparison directions, and sorting in the wrong order. The model produces code that reads convincingly but does not match the actual specification. This is why writing tests independently of the AI (rather than asking the AI to test its own output) is critical—the AI will tend to write tests that confirm its own assumptions, including its own mistakes. Human developers do the same thing, which is why many companies have a separate QA organization that performs independent testing.

Outdated library usage occurs because models are trained on code snapshots. The model may use a deprecated API, rely on a default that changed in a recent release, or follow a pattern that was idiomatic two versions ago but now triggers warnings or silent behavior changes. If AI-generated code produces deprecation warnings or behaves differently than the documentation suggests, check whether the model used an older version of the library's API.

Confident repetition of the same mistake is an AI-specific debugging trap. When you paste an error message back to the model and ask it to fix the code, the model will often produce a "fix" that looks different but contains the same underlying error—sometimes with more complexity layered on top. If the model has failed to fix a bug after two or three attempts, it is likely stuck in a local minimum of its own reasoning. Continuing to iterate rarely helps and consumes tokens. At that point, switch to a different strategy (described below).

Diagnostic Strategies

When AI-generated code fails, resist the instinct to immediately ask the AI to fix it. Instead, apply a structured diagnostic approach.

Isolate the AI-generated code from your existing code. If you followed the session hygiene practices in Section 5.3 (starting from a clean Git state and committing incrementally), you can use git diff to see exactly what the agent changed. Review the diff to determine whether the failure originates in the AI-generated code or in an interaction between the new code and your existing code. This distinction determines whether the fix belongs in the AI-generated portion or in how you integrated it.

Read the error yourself first. Before feeding a stack trace back to the model, read it and form your own hypothesis. Identify which file and line the error points to, whether that line is AI-generated or pre-existing, and what the error message actually says. This takes sixty seconds and prevents a common anti-pattern: pasting an error into the AI,

accepting a “fix” you don’t understand, hitting a new error, pasting that one in, and spiraling into a chain of patches that makes the code worse with each iteration.

Verify assumptions before iterating. When AI-generated code fails, check three things before asking the model to retry. First, do the imported packages exist and are they the versions the code expects? Run pip show, npm list, or your package manager’s equivalent to confirm. Second, does the API the code calls actually have the signature the code uses? Check the library documentation, not the model’s assertion. Third, does the code’s logic match your specification, or did the model solve a slightly different problem than the one you described?

Use the AI to explain, not just to fix. If you do not understand why a piece of AI-generated code is failing, ask the model to explain the code line by line rather than asking it to fix the error. The explanation often reveals a flawed assumption—such as the model believing a function returns a list when it returns an iterator, or assuming a library handles null values when it does not—that you can then address directly. This also builds the mental model you need to maintain the code after the AI session ends.

When to Abandon and Start Over

Not every AI-generated approach is worth salvaging. Recognizing when to stop iterating and start fresh is a judgment call, but the following signals are reliable indicators.

The fix-retry loop has exceeded three iterations without progress. If you have described the bug to the model three times and it has not produced a working fix—or each fix introduces a new problem—the model’s approach is likely flawed at a structural level, not just at the line level. Scrap the generated code, simplify your prompt, and try again from scratch. A fresh session with a clearer specification almost always outperforms a sixth attempt to patch a broken approach.

The generated code has become harder to understand than code you would write yourself. AI agents sometimes produce overly complex solutions—deeply nested logic, unnecessary abstractions, or unfamiliar patterns borrowed from the training data. If the code has reached a point where you cannot confidently explain what it does, it has failed the self-test from Section 2.3 regardless of whether it passes tests. Replace it with a simpler approach you understand, even if that approach is less “clever.”

The bug is in the interaction between generated code and your existing codebase, not in the generated code itself. AI agents lack full context about your system’s runtime behavior, deployment environment, and implicit conventions. When the bug arises from how the generated code interacts with parts of the system the model could not see, iterating with the model is unlikely to help. Debug the integration yourself, using the model only as a reference for explaining what its code was intended to do.

6. Git Worktrees for AI Concurrency

Git worktrees are one of Git's most underutilized features, and they become essential infrastructure in AI-augmented development. A worktree is an additional working directory linked to the same repository, sharing its .git directory and object store. Unlike git clone, which creates a fully independent copy, worktrees let you check out multiple branches simultaneously in separate directories—and merge, rebase, or compare them without network operations. Figure 2 illustrates this structure: each agent operates in its own worktree directory on a dedicated branch, all sharing a single .git object store.

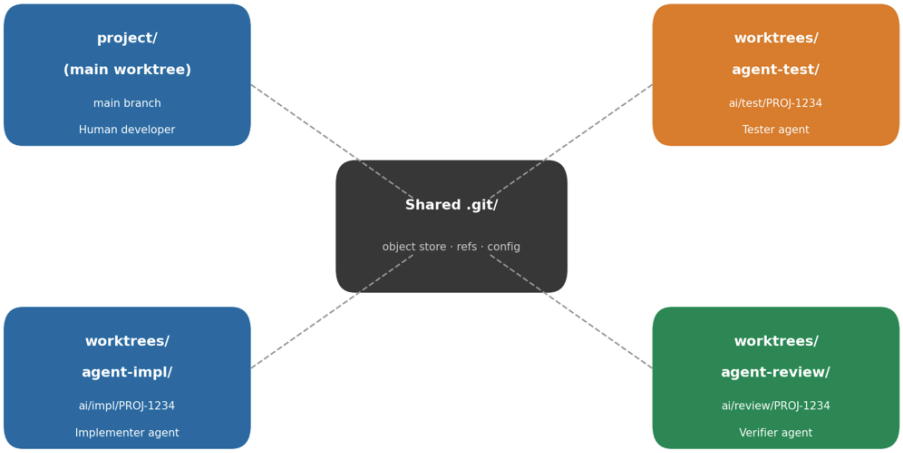

Figure 2: Git Worktree Isolation Model.

6.1 Why Worktrees Are Essential for AI Concurrency

When multiple AI agents operate on a codebase simultaneously, they must not interfere with each other's file modifications. Traditional branch-based workflows are inherently serial—only one branch can be checked out at a time in a single working directory. Worktrees solve this by giving each agent its own isolated directory on its own branch, all within the same repository. Without this isolation, concurrent agents writing to the same file on disk produce destructive conflicts: the last write wins, and the other agent's work is lost.

6.2 Branch-Per-Agent Strategy

The recommended strategy is to assign each agent a dedicated branch and a dedicated worktree. The branch naming convention should encode the agent's role, the task identifier, and a timestamp for uniqueness:

```
# Create a worktree for an implementation agent
git worktree add ../worktrees/agent-impl-PROJ-1234 \
  -b ai/impl/PROJ-1234-$(date +%s)

# Create a worktree for a testing agent
git worktree add ../worktrees/agent-test-PROJ-1234 \
  -b ai/test/PROJ-1234-$(date +%s)

# List all active worktrees
git worktree list
```

Each worktree is a fully functional working directory with its own checked-out branch. The agent operates exclusively within its assigned worktree directory. When the agent completes its task, the branch is merged (after validation) and the worktree is cleaned up.

6.3 Parallel Agent Execution and Tool Setup

The parallel execution model works as follows: the developer creates a worktree for each subtask and starts an agent in each one. Each agent works independently within its own isolated worktree — reading project files, generating diffs, applying changes, running tests, and committing results to its branch — without awareness of or interference with other agents working concurrently. The developer monitors progress across worktrees, enforces iteration caps, and initiates the merge sequence once all agents have completed their work.

In automated workflows, a script takes the place of the developer as coordinator. The script creates worktrees, spawns agents programmatically, monitors their output, enforces iteration caps by tracking attempt counts and terminating agents that exceed them, and triggers the merge sequence when all branches are complete. This is the same orchestration layer described in Section 7.2 in the context of multi-agent DAG execution — the mechanics are identical whether the agents are running sequentially or in parallel across worktrees. For simple projects, manual coordination is usually sufficient. For larger teams or workflows involving many concurrent agents, automation becomes necessary to manage the complexity reliably.

Setting Up Claude Code in a Worktree

Claude Code works naturally in worktrees because it operates on whatever directory you invoke it from. To run Claude Code in a worktree, simply change to the worktree directory before starting:

```
# Navigate to the worktree directory
cd ../worktrees/agent-impl-PROJ-1234

# Start Claude Code in this directory
claude

# Claude Code will read the CLAUDE.md from the worktree
```

```
# (inherited from the branch) and operate within this
# isolated filesystem.
```

To ensure persistent context is available in each worktree, your CLAUDE.md file should be committed to the repository (not gitignored). This way, every worktree branch inherits the project conventions automatically.

Setting Up Cursor in a Worktree

Cursor can open a worktree as a separate workspace. Use “File > Open Folder” and select the worktree directory. Because the worktree is a full working directory, Cursor will index it and the .cursorrules file (if committed to the repo) will apply automatically. To run multiple agents in parallel, open each worktree in a separate Cursor window:

```
# Open each worktree in a separate Cursor instance
cursor ../worktrees/agent-impl-PROJ-1234 &
cursor ../worktrees/agent-test-PROJ-1234 &
```

Example Prompt for Worktree-Based Agent Work

When initiating work within a worktree, provide context about the isolation:

```
You are working in an isolated Git worktree on branch
ai/impl/PROJ-1234. Your task is to implement input
validation for the user registration endpoint in
src/auth/register.ts.

Constraints:
- Only modify files in src/auth/.
- Write tests in tests/auth/.
- Run `npm test -- --filter auth` to validate.
- Do not modify any files outside your scope.
- Commit your changes when tests pass.
```

6.4 Cleanup Strategy

Worktrees accumulate over time and must be cleaned up to prevent disk bloat and branch pollution. Implement automated removal of worktrees whose branches have been merged, periodic pruning of stale worktrees older than a configurable threshold (e.g., 24 hours), and deletion of the associated branches:

```
# Remove a worktree after successful merge
git worktree remove ../worktrees/agent-impl-PROJ-1234

# Prune all worktrees whose directories were deleted
git worktree prune

# Delete the associated branch
git branch -d ai/impl/PROJ-1234-1708700000
```

7. Multi-Agent Orchestration Architecture

As AI-assisted development matures, single-agent workflows give way to multi-agent orchestration: systems where multiple specialized AI agents collaborate to complete complex tasks. This architecture mirrors the division of labor in human software teams, where different roles—architect, implementer, tester, reviewer—contribute complementary expertise to a shared objective.

7.1 Agent Roles and Responsibilities

The canonical multi-agent architecture defines four primary roles. The Planner decomposes a task into subtasks, assigns files, and defines acceptance criteria; it uses a cheap, fast model (Haiku tier) because planning requires structural reasoning rather than deep code generation. The Implementer generates code changes as unified diffs per the plan; it uses a mid-tier model (Sonnet) with a cap of five iterations. The Tester writes and executes tests, reporting pass/fail results; it also uses a mid-tier model and is capped at three iterations. The Verifier reviews the final diffs for correctness, security, and style compliance; it uses the most capable model (Opus) but only on the final output, keeping costs manageable with a single-pass (no iteration) approach.

Table 7: Multi-Agent Role Definitions

Role	Responsibility	Model	Scope	Iteration Cap
Planner	Decompose task, define acceptance criteria	Haiku (cheap)	Read-only	1
Implementer	Generate code changes as diffs	Sonnet (balanced)	Read/write in worktree	5
Tester	Write and run tests	Sonnet (balanced)	Read/write tests	3
Verifier	Review diffs for correctness & security	Opus (highest)	Read-only diffs	1

Each role is implemented by starting a Claude Code, Codex or Cursor Agent session in a dedicated worktree with a role-specific prompt that defines the agent's identity, scope, constraints, and output format. The key elements are consistent across all four roles: the prompt names the role explicitly ("You are acting as an IMPLEMENTER agent"), restricts the agent to its defined scope (e.g., "Only modify files listed in the plan"), enforces the iteration cap ("Stop after 5 failed attempts and report the error"), and specifies the expected output format. Appendix B provides complete, ready-to-use prompt templates for all four roles.

The four roles above are a starting point, not a ceiling. The DAG can be extended with additional roles wherever they add value. A Documenter, for example, could run after the Tester and before the Verifier, writing or updating architecture documentation that is committed to the repository and available to all agents via CLAUDE.md or AGENTS.md. A Product Owner role could sit upstream of the Planner, breaking large features — called epics in agile development — into smaller, well-scoped stories for the Planner to work from. Additional roles should follow the same pattern as the four canonical ones: assign a model tier appropriate to the task, define a clear scope, and set an iteration cap.

7.2 DAG-Based Execution Model

The multi-agent workflow is modeled as a directed acyclic graph (DAG) where nodes represent agent tasks and edges represent dependencies, as illustrated in Figure 3. The Planner node executes first, producing a structured plan document that defines subtasks, file assignments, and acceptance criteria. Implementer nodes depend on the Planner's output and may execute in parallel if the plan decomposes the task into independent subtasks. Tester nodes depend on their corresponding Implementer nodes — each Tester works in its own worktree which is branched from its Implementer's completed branch, ensuring it has access to the finished implementation without any worktrees sharing a directory. The Verifier node depends on all Tester nodes, works in its own worktree which is branched from the merge of all Tester branches, and executes only after all tests have passed.

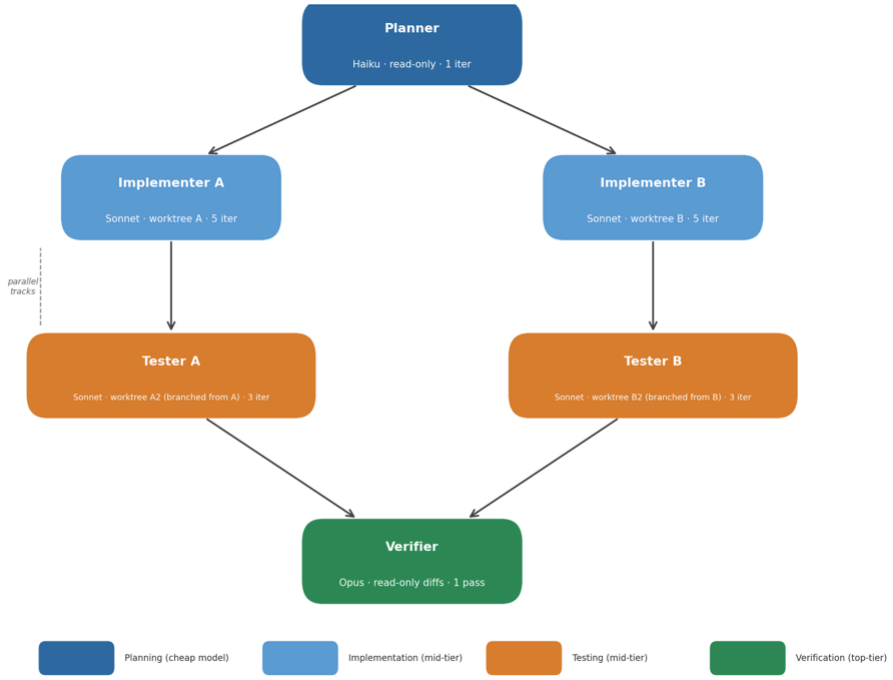

Figure 3: Multi-Agent DAG Execution Model. Nodes represent agent tasks; edges enforce dependency ordering. Parallel branches execute concurrently in separate worktrees.

It is worth noting that the collision risk that motivates worktree isolation applies specifically to agents running in parallel — in this architecture, the concurrent Implementer nodes. The Tester and Verifier roles are sequential by design: each begins only after the preceding agent has committed and exited, so there is no risk of conflicting file writes. For these roles, a fresh session on the same branch would provide the same clean snapshot benefit as a dedicated worktree. The branch-per-agent pattern is nevertheless recommended uniformly for two reasons: it produces a clean audit trail, making it easy to attribute exactly which changes each agent contributed; and it keeps the orchestration logic consistent regardless of whether agents run in parallel or in sequence, which simplifies automation. Developers working on smaller projects may reasonably omit dedicated worktrees for the Tester and Verifier stages, provided they commit and clear context between each agent handoff.

It is important to understand what the Planner does and does not do. The Planner produces the plan; it does not actively coordinate the agents that follow. Coordination — reading the plan, creating worktrees, spawning agents in the correct dependency order, enforcing iteration caps, and initiating the merge sequence — is the responsibility of an orchestration layer that sits outside the worktrees entirely. In simple workflows this can be a human developer following the plan manually. In automated workflows it is a script that parses the plan and manages agent execution programmatically. Either way, the DAG structure shown in Figure 3 makes the required ordering explicit: no agent starts until everything it depends on is complete, and steps with no dependency relationship can run in parallel.

This separation of planning from coordination is an important design principle. Mixing the two — asking the Planner agent to also monitor and trigger subsequent agents — would require it to maintain state across a long-running session, which degrades context coherence and increases cost. Keeping the Planner as a single-pass, read-only agent that produces a durable artifact (the plan document) makes the system more reliable and easier to debug.

7.3 Iteration Caps and Escalation

Each agent role has a defined iteration cap—the maximum number of attempts allowed before the task is escalated to human review. Without iteration caps, an agent encountering a persistent error will loop indefinitely, consuming tokens and producing no useful output. Iteration caps serve as a circuit breaker.

In Claude Code, you can enforce iteration caps through the CLAUDE.md file:

```
# CLAUDE.md (add to existing file)
## Iteration Policy
- Maximum 5 attempts per implementation subtask.
```

```
- If tests still fail after 5 attempts, STOP and report:
  the error, what you tried, and what you think is wrong.
- Do not attempt creative workarounds after the cap;
  escalate to the developer.
```

Beyond CLAUDE.md-level caps, programmatic orchestration should track both iteration count and total token consumption, terminating agents that exceed either threshold. In Cursor, where orchestration is manual, monitor conversation length and interrupt if the agent appears to be cycling.

Escalation to human review is not a failure of the system; it is a designed outcome. The multi-agent architecture handles the routine 80% of coding tasks autonomously, freeing developers to focus on the 20% that requires creative problem-solving, architectural decisions, or domain-specific knowledge.

8. Overwrite Prevention Techniques

One of the highest-severity risks in AI-augmented development is the unintended overwrite: an AI agent modifying or replacing code that another agent or a human developer has concurrently changed. A comprehensive overwrite prevention strategy employs multiple layers of defense, each targeting a different failure mode.

8.1 Branch and Worktree Isolation

Worktree isolation (Section 6) prevents physical file-level overwrites between concurrent agents. The remaining subsections address logical conflicts that can still emerge at merge time.

8.2 Read-Before-Write Prompt Enforcement

An additional directive for your .cursorrules, CLAUDE.md, or AGENTS.md file (see Section 5.2) is read-before-write enforcement: instruct the agent to read the current on-disk contents of a file before proposing modifications, ensuring that diffs are based on actual file state rather than a stale version from the agent's context window:

```
Before modifying any file, read its current contents
from disk. Base your changes on the actual file contents,
not on any previously cached version.
```

8.3 Diff-Only Enforcement and CI Rejection

As described in Section 5.1, diff-only workflows are enforced at the prompt level. At the CI level, add a complementary check: flag any commit from an AI agent branch where

more than a configurable percentage (e.g., 80%) of a file's lines changed in a single commit, and route it to human review rather than automated merge.

8.4 Infinite Loop Prevention

Iteration caps and token-consumption tracking (Section 7.3) prevent agents from entering infinite fix-retry cycles. Ensure every agent—whether orchestrated programmatically or run manually—is subject to these limits.

9. CI/CD Integration

AI-generated code must pass through the same CI/CD pipeline as human-authored code, and in many cases, it should pass through additional gates. The principle is non-negotiable: AI does not eliminate code review; it creates more code that requires review.

Test gating is the single most important deterministic checkpoint. Every AI-generated diff must pass the project's full test suite—unit tests, integration tests, and end-to-end tests—before it is eligible for merge. If an AI-generated change breaks an existing test, it is rejected regardless of how plausible the code looks. Test gating catches the most common LLM failure mode: code that is syntactically correct but semantically wrong.

Static analysis and linting enforce code style and catch common errors. AI-generated code is particularly prone to style drift: the model may produce code that is functionally correct but violates the project's naming conventions, import ordering, or formatting standards. Running ESLint, TypeScript's strict mode, Prettier, and language-specific linters on every AI-generated diff helps to ensure consistency.

Security scanning catches vulnerabilities that the model may introduce unwittingly: SQL injection vectors, insecure cryptographic practices, hardcoded credentials, or dependency versions with known Common Vulnerabilities and Exposures (CVEs). Tools like Snyk, Semgrep, or CodeQL should be run on every AI-generated diff. This is especially important because LLMs are trained on public code that includes both secure and insecure patterns.

Human review checkpoints are the final gate. AI-generated pull requests should be clearly labeled (e.g., with an ai-generated tag) so that reviewers know to apply heightened scrutiny. The reviewer's role shifts from evaluating authorship intent to evaluating correctness: does this diff do what it claims? Does it introduce hidden side effects? Is the generated code idiomatic? Human review is the ultimate backstop against failure modes that automated tools cannot catch, including subtle logic errors, inappropriate architectural decisions, and hallucinated API usage.

10. Conclusion

The integration of large language models into software development represents a genuine paradigm shift—not because AI can write code, but because it writes code probabilistically in an industry built on determinism. Every technique in this guide exists to bridge that gap: ensuring that probabilistic output is validated, isolated, and controlled before it enters the production codebase.

These guardrails form a layered defense. At the workflow level, diff-only enforcement and structured prompting constrain the format of AI output. At the isolation level, Git worktrees and branch-per-agent strategies prevent concurrent agents from corrupting each other's work. At the orchestration level, multi-agent architectures with iteration caps and DAG-based execution ensure that complex tasks proceed in a controlled, auditable manner. At the validation level, CI/CD pipelines with test gating, static analysis, security scanning, and human review catch errors before they reach production.

For students and junior developers entering the profession, the temptation to rely on AI-generated code without validation is strong, and the consequences of doing so—subtle bugs, security vulnerabilities, and codebase degradation—are often invisible until they compound into serious problems. The practitioners who will be most effective with AI tools are those who treat them as powerful but unreliable collaborators: valuable for their speed and breadth, but always requiring verification, containment, and oversight.

The tools and techniques described here will continue to evolve. Models will become more capable, context windows will grow, and multi-agent orchestration will become more sophisticated. But the fundamental principle—that probabilistic systems require deterministic guardrails—will remain. The engineering discipline required to build reliable software does not diminish with AI; it adapts. The practitioners who invest in building robust guardrail infrastructure today will be best positioned to exploit tomorrow's more capable models safely and productively.

11. Resources

The following resources provide official documentation, background research, and further reading relevant to the topics covered in this guide.

GitHub, "GitHub Copilot Documentation," GitHub Docs, 2026. [Online]. Available: https://docs.github.com/en/copilot

Anthropic, "Claude API Pricing," Anthropic, 2026. [Online]. Available: https://www.anthropic.com/pricing

Cursor, “Cursor Pricing and Plans,” Cursor, 2026. [Online]. Available: https://www.cursor.com/pricing

Anthropic, “Claude Code Documentation,” Anthropic, 2026. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code

OpenAI, "Codex Documentation," OpenAI Developers, 2026. [Online]. Available: https://developers.openai.com/codex/

Google, “Getting Started with Google Antigravity,” Google Codelabs, 2025. [Online]. Available: https://codelabs.developers.google.com/getting-started-google-antigravity

S. Bubeck et al., “Sparks of Artificial General Intelligence: Early Experiments with GPT-4,” Microsoft Research, arXiv:2303.12712, 2023.

M. Chen et al., “Evaluating Large Language Models Trained on Code,” OpenAI, arXiv:2107.03374, 2021.

Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” ACM Computing Surveys, vol. 55, no. 12, 2023.

Git Project, “git-worktree Documentation,” Git SCM, 2024. [Online]. Available: https://git-scm.com/docs/git-worktree

Anthropic, “Prompt Engineering Guide,” Anthropic Docs, 2025. [Online]. Available: https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering

J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022.

P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.

N. Perry et al., “Do Users Write More Insecure Code with AI Assistants?,” ACM CCS, 2023.

Google, “Build with Google Antigravity,” Google Developers Blog, Nov. 2025. [Online]. Available: https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform

Cursor, “Models,” Cursor Docs, 2026. [Online]. Available: https://cursor.com/docs/models

Brigham Young University, “Academic Honesty Policy,” BYU University Policies, 2025. [Online]. Available: https://policy.byu.edu/view/academic-honesty-policy

Brigham Young University, “Generative AI at BYU: Academic Integrity for Students,” BYU GenAI, 2025. [Online]. Available: https://genai.byu.edu/academic-integrity-for-students

Appendix A: Tool and Model Version Reference

The following tables provide a snapshot of tool and model versions current at the time of writing. AI tools and models evolve rapidly; readers should verify current versions against official documentation before making adoption decisions.

Table A.1: Tool Versions (February 2026)

Tool	Version / Release	Platform	Primary Interface
GitHub Copilot	Copilot X (2025–2026)	VS Code, JetBrains, Neovim extensions	Inline completion + Chat + Agent mode
Cursor	Cursor 2.x (Oct 2025+)	Standalone IDE (VS Code fork)	AI-native editor + Agent + Background agents
Claude Code	GA (May 2025+)	CLI, VS Code extension, Desktop app, Web IDE	Terminal-based agentic coding
Codex	CLI GA (Apr 2025+), Web GA (2025+)	CLI (local), ChatGPT Web, IDE Extension	Terminal CLI + Cloud Agent + IDE
Google Antigravity	Public Preview (Nov 2025)	Standalone IDE (VS Code fork)	Agent Manager + Editor + Browser

Table A.2: AI Model Versions (February 2026)

Model	Provider	API Identifier	Context	Used By
Claude Opus 4.6	Anthropic	claude-opus-4-6-20260205	1M (beta)	Claude Code, Cursor, Antigravity
Claude Sonnet 4.6	Anthropic	claude-sonnet-4-6-20260217	1M (beta)	Claude Code (default), Cursor
Claude Sonnet 4.5	Anthropic	claude-sonnet-4-5-20250929	200K	Claude Code, Cursor, Antigravity
Claude Haiku 4.5	Anthropic	claude-haiku-4-5-20251001	200K	API (cost-optimized tasks)
GPT-5.3-Codex	OpenAI	gpt-5.3-codex	128K	Copilot (default), Cursor
GPT-5	OpenAI	gpt-5-20260401	128K	Copilot, Cursor
Gemini 3 Pro	Google	gemini-3-pro	1M	Antigravity (default), Cursor
Gemini 3 Deep Think	Google	gemini-3-deep-think	1M	Antigravity (reasoning)
Composer-1	Cursor	cursor-composer-1	N/A	Cursor (proprietary)

Date of version information: February 23, 2026. All versions and pricing are subject to change. Consult official documentation for current information.

Appendix B: Multi-Agent Role Prompt Templates

The following prompt templates implement the four agent roles defined in Section 7.1 (Table 6). Each prompt is designed for use with Claude Code or Codex CLI but can be adapted for Cursor Agent or other tools. Customize the task description, file paths, test commands, and acceptance criteria to match your project.

Planner Prompt

```
You are acting as a PLANNER agent. Your task is to
decompose the following feature request into subtasks.

Feature: Add rate limiting to the /api/auth/* endpoints.

Output a structured plan with:
1. List of files to modify
2. Subtask descriptions
3. Acceptance criteria for each subtask
4. Dependency ordering (which subtasks must complete first)

Do NOT write any code. Only produce the plan.
```

Implementer Prompt

```
You are acting as an IMPLEMENTER agent working in an
isolated worktree. Follow the plan below exactly.

[Paste planner output here]

Rules:
- Only modify files listed in the plan.
- Produce minimal diffs; do not regenerate entire files.
- After each change, run the existing test to confirm no regressions: `npm test`
- Do not write new tests; that is the Tester agent’s responsibility.
- Stop after 5 failed attempts and report the error.
- When all existing tests pass, commit your changes and report completion.
```

Tester Prompt

```
You are acting as a TESTER agent working in your own isolated worktree, branched from
the implementer’s completed branch. Your task is to validate the rate limiting
implementation against the acceptance criteria below.

[Paste acceptance criteria from planner output here]

Rules:
- Only create or modify files in tests/auth/.
- Use the Vitest framework, consistent with the project's test configuration.
- Write tests that verify each acceptance criterion explicitly: test correct rate
```

```
  limit thresholds, verify appropriate error responses (HTTP 429), and confirm that
  legitimate requests below the threshold are not blocked.
- After writing tests, run them: `npm test -- --filter auth`
- Report results as: total tests, passed, failed, and for each failure: test name and
  error message.
- Do NOT modify any implementation code. If a test fails due
  to an implementation defect, report the failure with the
  expected vs. actual behavior.
- Stop after 3 attempts. If tests still fail after 3 attempts, report which acceptance
  criteria are not met and why.
```

Verifier Prompt

```
You are acting as a VERIFIER agent working in your own isolated worktree, branched from
the merge of all Tester Branches. You are the final gate before human review. Your task
is to review the combined diffs from the rate limiting implementation across both the
Implementer and Tester branches.


[Paste the combined git diff output from the Implementer
and Tester branches here]

Review the diffs and produce a structured report covering:

1. Correctness: Does the implementation match the plan's
   acceptance criteria? Are there logic errors, off-by-one
   mistakes, or unhandled edge cases?
2. Security: Are there vulnerabilities such as bypass vectors,
   race conditions in the rate counter, hardcoded secrets,
   or missing input validation?
3. Style and conventions: Does the code follow the project's
   conventions (named exports, Zod validation, Result<T,E>
   error handling) as defined in CLAUDE.md or AGENTS.md?

Rules:
- Reference specific files and line numbers for every finding.
- Do NOT modify any code. This is a read-only review.
- Classify each finding as: BLOCKING (must fix before merge),
  WARNING (should fix, not a merge blocker), or NOTE
  (suggestion for improvement).
- Conclude with a verdict: PASS (no blocking findings),
  CONDITIONAL PASS (blocking findings are minor and
  well-defined), or FAIL (significant issues requiring
  re-implementation).
```

Appendix C: Glossary of Key Terms

Agentic. Describes an AI tool or mode that can take multi-step actions autonomously—reading files, running commands, modifying code, and iterating on errors—without requiring human approval between each step. Claude Code and Cursor's Agent mode are agentic; Copilot's inline autocomplete is not.

API (Application Programming Interface). A set of rules that allows software programs to communicate with each other. In this guide, "API" most often refers to the HTTP endpoints provided by model providers (Anthropic, OpenAI, Google) that accept prompts and return model outputs, billed per token.

Blast radius. The scope of potential damage from a bad change. A single-line autocomplete suggestion has a small blast radius; a multi-file agentic refactor has a large one. Guardrails like diff-only workflows and worktree isolation exist to limit blast radius.

CI/CD (Continuous Integration / Continuous Deployment). An automated pipeline that builds, tests, lints, and deploys code whenever changes are pushed to a repository. In this guide, CI/CD serves as the final deterministic gate for AI-generated code before it reaches the main branch.

Circuit breaker. A control mechanism that halts an automated process when a predefined failure threshold is reached. In this guide, iteration caps serve as circuit breakers: when an agent exceeds its maximum attempts, the task is escalated to human review rather than allowed to loop indefinitely. See Section 7.3.

Codebase index. A searchable representation of an entire repository's files, symbols, and structure that an AI tool builds locally. Tools with a codebase index (like Cursor) can reason across multiple files; tools without one (like basic Copilot) are limited to what is currently open in the editor.

Context window. The maximum amount of text (measured in tokens) that a model can process in a single request, including both the input prompt and the output it generates. When a conversation or codebase exceeds the context window, the model loses access to earlier content, which can cause it to forget instructions, repeat itself, or produce inconsistent output.

DAG (Directed Acyclic Graph). A graph structure where edges have direction and no cycles exist—meaning you can follow the edges forward but never loop back to where you started. In Section 7, the multi-agent workflow is modeled as a DAG: the Planner must finish before Implementers start, Testers wait for their Implementer, and the Verifier runs last.

Deterministic. Producing the same output given the same input, every time. Traditional software tools (compilers, linters, test suites) are deterministic. LLMs are not, which is the core tension this guide addresses.

Diff (unified diff). A text format that shows exactly which lines were added, removed, or changed in a file. Diff-only workflows instruct the AI to produce changes in this format rather than regenerating entire files, making output reviewable, composable, and reversible.

Ghost text. The faded, inline code suggestion that appears in your editor as you type, offered by tools like GitHub Copilot. You accept it with a keystroke (usually Tab) or dismiss it by continuing to type.

Guardrail. Any deterministic structure that constrains AI output before it enters the production codebase. Examples include diff-only enforcement, iteration caps, test gating, worktree isolation, and human review checkpoints.

Hallucination. When an AI model generates output that is plausible-looking but factually wrong—such as calling a function that does not exist, importing a package that was never published, or citing an API endpoint with an incorrect signature. Hallucinations are particularly dangerous in code because they can pass syntax checks and linting while being semantically incorrect.

Iteration cap. A predefined maximum number of attempts an AI agent is allowed before the task is escalated to human review. Without iteration caps, an agent encountering a persistent error will loop indefinitely, consuming tokens and producing no useful output. See Section 7.3.

LLM (Large Language Model). A neural network trained on large amounts of text that generates output by predicting the most likely next token in a sequence. GPT-5, Claude, and Gemini are all LLMs. In this guide, LLMs are the engines behind AI coding tools.

Linting. Automated analysis that checks code for style violations, common errors, and suspicious patterns without executing it. Tools like ESLint (JavaScript/TypeScript), Pylint (Python), and Prettier (formatting) are linters. Linting is one layer of the CI/CD validation pipeline for AI-generated code.

Model tiering. The practice of routing tasks to different AI models based on complexity and cost: cheap, fast models (like Haiku) for planning and simple tasks; mid-tier models (like Sonnet) for implementation; and expensive, capable models (like Opus) only for final verification. This reduces overall cost by 40–60%.

Monorepo. A single Git repository that contains multiple projects, packages, or services. Monorepos are common in modern development and are relevant to AI tooling because they increase the codebase size that must fit within the context window.

Probabilistic. Producing output that varies based on sampling from a probability distribution. LLMs are probabilistic: the same prompt can produce different outputs on different runs, depending on temperature and sampling settings. This is the opposite of deterministic.

Prompt caching. A feature (offered by Anthropic and others) that caches the prefix of repeated prompts so that only the new portion is billed at full price on subsequent requests. This can reduce input token costs by up to 90% when the system prompt and codebase context are re-sent with each iteration.

Pull request (PR). A request to merge code changes from one branch into another, typically accompanied by a review process. In AI-augmented workflows, AI-generated pull requests should be labeled (e.g., with an ai-generated tag) so reviewers know to apply heightened scrutiny.

Seat-based pricing. A subscription model that charges a flat monthly fee per user regardless of usage. GitHub Copilot's Pro plan ($10/month per seat) is an example. The advantage is predictable cost; the disadvantage is paying the same amount during low-usage periods.

Static analysis. Examining code without executing it to find bugs, security vulnerabilities, or style issues. Linting is one form of static analysis; security scanners like Semgrep and CodeQL are others. Static analysis is a deterministic check in the CI/CD pipeline.

Temperature. A parameter that controls the randomness of an LLM's output. A temperature of 0 makes the model nearly deterministic (always choosing the most probable next token); higher temperatures increase variety and creativity but also increase the chance of hallucination. Most coding tasks benefit from low temperature settings.

Token. The basic unit of text that an LLM processes. A token is roughly three-quarters of a word in English—the word "programming" is two tokens, and a typical line of code is 10–15 tokens. Tokens are the unit of billing for API-priced models: you pay per million input tokens (your prompt) and per million output tokens (the model's response).

Token-based pricing. A billing model that charges per token consumed. Anthropic's API pricing is the canonical example: Claude Sonnet 4.5 costs $3 per million input

tokens and $15 per million output tokens. Cost scales directly with usage, making it efficient for light use but potentially expensive for iterative agentic workflows.

Worktree (Git worktree). An additional working directory linked to the same Git repository, sharing its .git directory and object store. Unlike git clone, worktrees let you check out multiple branches simultaneously in separate directories on the same machine. In this guide, worktrees provide the isolation layer that allows multiple AI agents to work concurrently without overwriting each other's files. See Section 6.